```
        String authorizeRequest) throws Exception {

    var secureRandom = new java.security.SecureRandom();
    var encoder =
java.util.Base64.getUrlEncoder().withoutPadding();

    var verifierBytes = new byte[32];     #A
    secureRandom.nextBytes(verifierBytes);     #A
    var verifier = encoder.encodeToString(verifierBytes);
#A

    request.session(true).attribute("verifier", verifier);
#B

    var sha256 =
java.security.MessageDigest.getInstance("SHA-256");    #C
    var challenge = encoder.encodeToString(     #C
            sha256.digest(verifier.getBytes("UTF-8")));
#C

    return authorizeRequest +
        "&code_challenge=" + challenge +     #D

        "&code_challenge_method=S256";     #D
}
```

#A Create a random code verifier string.
#B Store the verifier in a session cookie or other local storage.
#C Create a code challenge as the SHA-256 hash of the code verifier string.
#D Include the code challenge in the redirect to the AS authorization endpoint.

Later, when the client exchanges the authorization code at the token endpoint, it sends the original (unhashed) code